

Online Electric Vehicle Routing under Time-Dependent Traffic and Charging Delay: A Controlled Comparison of Distance-Oriented and Time-Aware Policies

Trung Duc Tran¹, Nguyen Toan Thang², Doanh Nguyen Ngoc¹

¹*Center for Environmental Intelligence, VinUniversity, Hanoi, Vietnam*

²*Hanoi University of Science and Technology, Hanoi, Vietnam*

Corresponding author: Doanh Nguyen Ngoc, doanh.nn@vinuni.edu.vn

Abstract

This paper studies a single-vehicle electric vehicle routing problem with time-dependent travel time and charging-station delay. The problem is modeled as an online routing task in which route quality depends jointly on geometric distance, traffic-dependent travel time, charging delay, and battery feasibility. We compare two online policies within the same execution simulator and iterated local search framework: a distance-oriented heuristic that follows a geometric customer plan with nearest-station repair, and a delay-aware rollout policy that evaluates candidate customer and charging actions by predicted remaining completion time. Computational experiments on eight synthetic datasets suggest that the delay-aware policy tends to achieve lower total completion time than the distance-oriented baseline on the tested benchmark. The results also indicate that the delay-aware policy can travel farther while still finishing earlier, suggesting that under time-varying traffic and charging delay, shorter distance is not always a reliable proxy for shorter operational time in the tested setting. To separate tuned best-case evidence from fixed-setting evidence, we report both a per-instance rollout-window sweep and a three-seed study at the tuned W^* values. In the fixed-setting study, Algorithm 2 achieves lower realized completion time in all 24 paired runs, with a bootstrap 95% confidence interval for mean ΔT of $[-55.23, -31.97]$ minutes.

Keywords: electric vehicle routing, time-dependent travel time, charging-station delay, online routing, rollout heuristic, iterated local search

1 Introduction

Electric vehicle routing has become an important research topic in transportation science and sustainable logistics because route planning for electric fleets must jointly account for customer service, energy consumption, and charging requirements. In contrast to conventional vehicle routing, the limited driving range of electric vehicles and the need for intermediate recharging create a strong coupling between routing and energy management. This coupling becomes more challenging when traffic conditions vary over time and charging stations are not instantaneously available.

The broader routing literature provides the methodological background for this problem. Classical VRP studies established the foundations of route construction and improvement [7, 4, 31, 17], while EVRP research showed that battery constraints and recharging decisions fundamentally alter route structure [5, 29, 13, 21]. More recent work has also highlighted the importance of time-dependent travel conditions, dynamic decision-making, and charging-station waiting effects [9, 25, 15, 14]. Together, these studies suggest that geometric distance alone is often an incomplete proxy for actual operational perfor-

mance.

This issue is especially relevant in online execution settings. A route that is short in distance may still finish late if it encounters peak-period congestion or arrives at a charging station when waiting delay is high. Conversely, a longer route may finish earlier if it is better synchronized with favorable traffic and charging conditions. In this paper, the term online refers to this sequential execution-and-replanning setting under time-dependent travel and charging conditions, rather than to customer-request revelation.

Motivated by this gap, we study a single-vehicle electric vehicle routing problem with time-dependent travel times and time-dependent charging-station delay. The transportation system contains customers, charging stations, and a depot. Travel times on a subset of arcs vary by traffic regime, charging stations may impose arrival-time-dependent waiting, and each move must satisfy battery-feasibility constraints with a safety reserve. Within this setting, we ask whether a policy that explicitly evaluates downstream completion time can yield lower realized completion time than a geometry-driven policy that mainly follows customer order and repairs infeasibility by nearest-station charging actions in a controlled time-

dependent execution setting.

To answer this question, we compare two online routing algorithms under the same execution model and the same ILS-based search template. The first algorithm is a distance-oriented baseline that follows a customer plan derived from geometric distance and inserts charging detours through nearest reachable stations when necessary. The second algorithm is a delay-aware rollout policy that evaluates candidate customer and charging actions by predicted remaining completion time. The paper therefore provides a controlled comparison between two distinct online decision philosophies under a unified time-dependent EV execution environment. Accordingly, the purpose of the paper is not to claim broad empirical superiority over the wider EVRP literature, but to compare two online decision philosophies within a common simulator-based framework.

The paper makes three main contributions. First, it formulates a stylized time-dependent EV execution setting that combines battery-constrained routing, time-varying travel conditions, and queue-aware charging delays. Second, it develops a controlled comparison within a shared framework between a distance-oriented policy and a delay-aware rollout policy under a common simulation and search environment. Third, it provides benchmark evidence that, on the tested synthetic instances, shorter geometric distance is not necessarily a reliable proxy for shorter realized completion time once temporal traffic exposure and charging delay are taken into account. The computational study includes both a tuned batch comparison and a fixed-setting three-seed study; in the latter, the delay-aware rollout policy yields lower completion time in all 24 paired runs on the tested synthetic benchmark, although this improvement comes with substantially higher computational cost.

The remainder of the paper is organized as follows. Section 2 reviews the most relevant literature on electric vehicle routing, time-dependent and dynamic routing, charging-station congestion, and online sequential decision methods. Section 3 presents the problem formulation and the dynamic execution model. Section 4 describes the two online routing algorithms and their common search framework. Section 5 reports the computational results and discusses the main empirical findings. Finally, Section 6 concludes the paper and outlines future research directions.

2 Literature Review

The problem studied in this paper lies at the intersection of several related research streams, including classical vehicle routing, electric vehicle routing, time-dependent and dynamic routing, charging-station congestion modeling, and online heuristic decision-making. Since our setting combines battery constraints, time-varying travel speeds,

charging delays, and heuristic online control, the most relevant literature is broader than static EVRP alone. It also includes time-dependent routing, dynamic routing, queue-aware charging models, and rollout or simulation-based sequential decision methods.

2.1 Time-dependent and dynamic routing

Building on the classical VRP and EVRP foundations already noted in the introduction, the literature most directly related to the present paper concerns routing in environments that evolve over time. Early time-dependent routing models appeared in [19], while later work emphasized departure-time-sensitive travel-speed formulations and more realistic temporal consistency properties [11]. Further developments studied benchmark instances, efficient heuristics, and exact or decomposition-based methods for time-dependent routing [9, 6]. More recent surveys confirm that routing over time has become a substantial research area of its own [22, 1].

Closely related is the dynamic vehicle routing literature, where the operating environment evolves during route execution and decisions must therefore be updated online. Important reviews include Pillac et al. [25], Psaraftis, Wen, and Kontovas [26], Rios et al. [27], and Zhang and Van Woensel [34]. A recurring conclusion in this literature is that purely offline plans become increasingly fragile when the environment changes over time, and that fast online heuristics, receding-horizon approaches, and approximate decision policies often provide a more appropriate trade-off between solution quality and response speed.

A third closely related stream concerns charging-station modeling. Many EVRP formulations assume that charging stations are immediately available, which simplifies the routing problem but ignores congestion. More recent studies therefore incorporate waiting or queueing at charging facilities. In particular, Keskin, Laporte, and Çatay [15] introduced time-dependent waiting at recharging stations, showing that station choice should depend on temporal conditions as well as distance. Later work extended this perspective through simulation-based treatment of stochastic waiting times [12, 14]. For the present paper, this stream matters because time-varying charging delay can make nearest-station decisions suboptimal in realized completion time.

2.2 Heuristic online control, rollout, and positioning of the present work

Because EVRP variants quickly become difficult once routing, charging, and temporal interactions are modeled jointly, heuristics and metaheuristics dominate practical solution approaches. In the broader routing literature, variable neighborhood search [20], adaptive large neighborhood search [28], iterated local search [18], and unified

Table 1: Representative studies most closely related to the present work.

Year	Reference	Travel environment	Station delay	Main method	Decision setting
2012	Figliozzi [9]	Time-dependent	None	Heuristic	Offline
2013	Dabia et al. [6]	Time-dependent	None	Branch-and-price	Offline
2014	Schneider et al. [29]	Static	None	Hybrid heuristic	Offline
2016	Keskin and Çatay [13]	Static	None	ALNS	Offline
2017	Montoya et al. [21]	Static	None	Hybrid metaheuristic	Offline
2018	Verma et al. [32]	Static	None	Heuristic	Offline
2018	Bertazzi and Secomandi [2]	Dynamic / stochastic	None	Rollout	Online
2019	Froger et al. [10]	Static	None	Exact / heuristic	Offline
2019	Keskin et al. [15]	Static	Expected waiting	Heuristic	Offline
2021	Keskin et al. [14]	Static	Stochastic waiting	Simulation-based heuristic	Hybrid
2024	Zhou et al. [35]	Time-dependent	None	MILP + heuristic	Offline
2025	Erdelić et al. [8]	Time-dependent	None	MILP + ALNS	Offline
This paper		Time-dependent	Time-varying delay	Delay-aware rollout policy	Online

modern search frameworks [33] have all shown that high-quality routing solutions often emerge from repeated perturbation, repair, and local improvement rather than from a single one-shot construction rule. The EVRP literature follows the same direction, as seen in [29, 13, 21, 10], where route order, charging decisions, and temporal feasibility are handled jointly through adaptive search.

The stream most directly related to our delay-aware policy is the literature on rollout, approximate dynamic programming, and simulation-based sequential control. In approximate dynamic programming, rollout is commonly interpreted as a one-step policy-improvement mechanism in which candidate actions are evaluated using a simulated or estimated downstream cost-to-go [3]. In routing contexts, this principle has been explored in stochastic and dynamic settings, for example in [30, 24, 2]. Closely related ideas also appear in recent learning-based sequential routing methods, including reinforcement-learning policies and attention-based neural routing heuristics [23, 16]. Although these approaches differ in how the decision rule is obtained, they share the same high-level perspective that routing decisions should internalize downstream consequences rather than rely only on static geometric order.

Table 1 summarizes representative studies closest to our setting. The main gap is not the absence of heuristic methods in EVRP, but the limited controlled comparison between geometry-oriented and time-aware online policies in a common EV execution environment with both time-varying travel speed and charging delay. Our paper addresses this gap by comparing, under the same simulator and ILS framework, a distance-oriented policy with nearest-station repair against a delay-aware rollout policy

that evaluates actions by predicted remaining completion time.

3 Problem Formulation

For ease of reference, Table 2 summarizes the main notation used throughout this section.

3.1 Problem setting and execution state

We study a single-vehicle electric vehicle routing problem with time-dependent travel time and time-dependent charging-station delay. The transportation system is represented by a complete directed graph

$$G = (V, E), \quad V = \{0\} \cup C \cup S.$$

The vehicle starts from the depot, serves all customers, may visit charging stations when necessary, and finally returns to the depot.

For each ordered pair $(i, j) \in V \times V$, let d_{ij} denote the travel distance from node i to node j . Energy consumption on arc (i, j) is proportional to distance:

$$e_{ij} = \rho d_{ij},$$

where ρ is the constant energy-consumption rate. Under the full-recharge policy adopted in the simulator, the charging time from battery level b is

$$c(b) = \frac{Q - b}{\gamma}.$$

Table 2: Core notation used in Section 3.

Symbol	Meaning
0	depot
C, S, V	customer set, charging-station set, and node set, with $V = \{0\} \cup C \cup S$
d_{ij}, e_{ij}	travel distance and energy consumption on arc (i, j)
$\tau_{ij}(t)$	travel time on arc (i, j) when departing at time t
$w_s(t)$	waiting time at station s when arriving at time t
s_j	service time at customer $j \in C$
Q, R, γ	battery capacity, safety reserve, and charging rate
$x = (i, t, b, U, \pi)$	execution state: current node, time, battery level, unserved-customer set, and current customer plan
$r = (v_0, \dots, v_m)$	executed depot-to-depot route generated online
$T(r), D(r)$	realized completion time and traveled distance of route r

Travel time is time-dependent. Let $\tau_{ij}(t)$ denote the travel time on arc (i, j) when the vehicle departs at time t . In the execution model, some edges are time-sensitive and follow the current traffic regime, while non-congested edges are traversed under the uncongested regime. Hence the realized time cost of an arc depends not only on geometry but also on departure time.

Charging stations are also time-dependent. For each station $s \in S$, let $w_s(t)$ denote the waiting time incurred when the vehicle arrives at station s at time t . Thus, station choice affects not only energy feasibility but also realized completion time through queueing delay.

At each decision instant, the vehicle state is represented by

$$x = (i, t, b, U, \pi).$$

The plan π contains customers only; charging stations are not part of the initial customer sequence and are inserted online during execution when needed. The plan π should therefore be understood as a customer-service sequence rather than as a complete depot-to-depot route.

If the vehicle departs from node i at time t with battery b and moves to node j , then the pre-processing arrival state is

$$t^{\text{arr}} = t + \tau_{ij}(t), \quad b^{\text{arr}} = b - e_{ij}.$$

If $j \in C$, then service time s_j is added after arrival. If $j \in S$, then waiting time $w_j(t^{\text{arr}})$ and charging time $c(b^{\text{arr}})$ are added after arrival, and the battery is reset to full capacity. If $j = 0$, no additional processing time is incurred.

Therefore, route quality is determined jointly by geometry, time, and battery state, and shortest-distance routing is not generally equivalent to shortest-time routing.

3.2 Executed route, objective, and feasibility

A realized solution is an executed route trace

$$r = (v_0, v_1, \dots, v_m),$$

where $v_0 = v_m = 0$, every customer in C is visited exactly once, and charging stations in S may be visited multiple times. Because charging decisions are inserted online and stations may be revisited, the route is not fixed in advance as a static depot-to-depot sequence. Instead, it is generated sequentially from the evolving state $x = (i, t, b, U, \pi)$.

For each visit index k , let a_k denote the arrival time at node v_k , let q_k denote the battery level upon arrival at v_k , and let δ_k denote the departure time from v_k after any service or charging performed there. The operational objective is to minimize the realized completion time, that is, the time at which the vehicle returns to the depot after serving all customers and performing any required charging actions. For an executed route r , the completion time can be written as

$$\begin{aligned} T(r) = & \sum_{k=0}^{m-1} \tau_{v_k v_{k+1}}(\delta_k) \\ & + \sum_{k: v_k \in C} s_{v_k} + \sum_{k: v_k \in S} w_{v_k}(a_k) \\ & + \sum_{k: v_k \in S} c(q_k), \end{aligned}$$

where m is the number of traveled arcs in the executed route. The first term is total travel time, the second is customer service time, and the last two terms are station waiting and charging times. The optimization problem is therefore

$$\min_{r \in \mathcal{R}} T(r),$$

where $\mathcal{R}$ is the set of all feasible executed depot-to-depot EV routes under the battery and time propagation rules defined below. Accordingly, the optimization target is the realized executed route r , whereas the customer plan π is only an internal decision structure used to generate that route online. In addition to the operational objective, the experimental study also reports a distance-oriented metric:

$$D(r) = \sum_{k=0}^{m-1} d_{v_k v_{k+1}}.$$

For Algorithm 1, this quantity is used as the optimization objective inside the ILS layer. This distinction is useful because it separates geometric efficiency from true operational performance under time-varying traffic and charging delay.

Feasibility is defined by customer completion, battery safety, temporal consistency, and route completion. First, every customer must be served exactly once, so the terminal state must satisfy

$$U = \emptyset.$$

Second, any direct move from the current node i to a candidate next node j is feasible only if the battery is sufficient for the trip while preserving the minimum reserve:

$$b \geq e_{ij} + R.$$

This is the direct-feasibility condition.

However, direct feasibility alone is not sufficient in the execution model. If $j \in C$, the move is accepted only if it also passes a post-arrival survivability check. Specifically, after traveling to customer j and serving that customer, the vehicle must still be able to reach at least one charging station while preserving the reserve. Writing

$$b_j^+ = b - e_{ij},$$

a customer move $i \rightarrow j$ is considered safe only if

$$\exists s \in S \text{ such that } b_j^+ \geq e_{js} + R.$$

This condition prevents locally feasible but immediately trapping moves.

Time propagation follows the route dynamics. If node j is visited next from state (i, t, b, U, π) , then the next decision time becomes

$$t^+ = \begin{cases} t + \tau_{ij}(t) + s_j, & j \in C, \\ t + \tau_{ij}(t) + w_j(t + \tau_{ij}(t)) + c(b - e_{ij}), & j \in S, \\ t + \tau_{ij}(t), & j = 0. \end{cases}$$

The corresponding next battery state is

$$b^+ = \begin{cases} b - e_{ij}, & j \in C \cup \{0\}, \\ Q, & j \in S. \end{cases}$$

If $j \in C$, then j is removed from both U and the current customer plan π . Charging stations may be revisited, since multiple charging actions may be required before all customers are served and the vehicle returns to the depot.

After all customers have been served, the same feasibility logic is applied to the return-to-depot phase. In particular, the vehicle may return directly to the depot only if the depot is safely reachable with reserve; otherwise, an intermediate charging detour is required. Hence feasibility includes not only serving all customers, but also completing the final depot return under the same battery and time-update rules.

Overall, a feasible solution in this paper is an executed depot-to-depot route that satisfies customer completion, battery safety, temporal consistency, and final return feasibility. This completes the problem formulation used in the algorithmic section.

4 Online Routing Algorithms

This section describes the algorithmic framework used in the unified implementation. The code does not construct a fixed route once and for all. Instead, it first builds an initial customer plan and then evaluates that plan through an online execution simulator that inserts charging decisions dynamically. On top of this simulator, both methods use the same iterated local search (ILS) layer to improve customer order. The difference between the two methods is therefore not the search scaffold itself, but the online policy used to execute and evaluate a candidate plan.

4.1 Shared Framework

Both algorithms use the execution state $x = (i, t, b, U, \pi)$ defined in Section 3, together with the same battery-feasibility rules, time propagation, station-wait model, and full-charge action at charging stations.

The framework has three layers. First, an initial customer-only plan is built by solving a depot-anchored TSP on the geometric distance matrix. Charging stations are excluded from this plan because station visits are inserted online only when battery feasibility requires them. Second, a route execution policy simulates how the vehicle actually moves through customers and charging stations under time-varying traffic and waiting conditions. Third, ILS modifies customer order and repeatedly re-evaluates the resulting plan through the same simulator.

Direct movement from the current node i to a customer j follows the direct-feasibility and post-arrival survivability conditions introduced in Section 3.2. We do not repeat those rules here; the purpose of this section is to explain how the two online policies use the same execution model differently.

The shared ILS layer operates on customer permutations only. Let $P = [0, c_1, \dots, c_n]$ denote a depot-anchored customer plan. For a given evaluation function $\mathcal{E}(P)$ and objective function $f(\mathcal{E}(P))$, the search proceeds as follows. It first applies local search to the current plan. Each local-search round samples random neighbors generated by one of three move types: 2-opt, swap, or relocate. The first improving neighbor is accepted, and the local search restarts from that new incumbent. If no improvement is found within the sampled neighborhood budget, the local-search phase stops.

After local search, the outer ILS loop perturbs the current customer sequence by applying several random neighborhood moves, runs local search again on the perturbed plan, and then applies an acceptance rule. Improving candidates are always accepted. Non-improving candidates may still be accepted with a temperature-controlled probability

$$p = \exp\left(-\frac{\Delta_{\text{rel}}}{T}\right),$$

Algorithm: Shared ILS Layer

Require: Initial customer plan P_0 , evaluator $\mathcal{E}$, objective f , search budgets

Ensure: Best customer plan and executed-route result

```
1:  $(P_{\text{cur}}, r_{\text{cur}}) \leftarrow \text{LOCALSEARCH}(P_0, \mathcal{E}, f)$ 
2:  $(P_{\text{best}}, r_{\text{best}}) \leftarrow (P_{\text{cur}}, r_{\text{cur}})$ 
3:  $T \leftarrow T_0$ 
4: for each ILS iteration do
5:    $\tilde{P} \leftarrow$  perturb the customer order of  $P_{\text{cur}}$ 
6:    $(P', r') \leftarrow \text{LOCALSEARCH}(\tilde{P}, \mathcal{E}, f)$ 
7:    $\Delta \leftarrow f(r') - f(r_{\text{cur}})$ 
8:   if  $\Delta \leq 0$  then
9:     accept  $(P', r')$  as current
10:  else
11:    accept with temperature-based probability
12:  end if
13:  if  $f(r') < f(r_{\text{best}})$  then
14:    update best solution
15:  end if
16:  cool temperature  $T$ 
17: end for
18: return  $(P_{\text{best}}, r_{\text{best}})$ 
```

where Δ_{rel} is the objective increase relative to the current solution and T is a gradually cooled temperature parameter. This gives the search a controlled way to escape local minima while still favoring better plans.

The implementation therefore compares two algorithmic combinations under a common search scaffold:

- Algorithm 1: distance-oriented online evaluator + distance-based ILS objective,
- Algorithm 2: delay-aware rollout online evaluator + time-based ILS objective.

Because the simulator and ILS operators are shared, the empirical gap is more cleanly interpreted as arising from the different evaluator-objective combinations used within the same execution and search scaffold.

4.2 Algorithm 1: Distance-Oriented Evaluation with ILS

Algorithm 1 is not merely a nearest-neighbor heuristic. In the unified implementation, it is a two-level method: a distance-based customer plan is first constructed, and that plan is then improved by ILS. What distinguishes Algorithm 1 is the evaluation policy used inside the simulator.

Given a current plan π , Algorithm 1 always tries to serve the first remaining customer in that plan. If that customer is directly feasible and the post-service survivability check is satisfied, the vehicle moves directly to that customer. Otherwise, the policy inserts a charging detour.

Charging-station choice in Algorithm 1 is purely geometric. If the vehicle is not currently at a station and

Algorithm 1 Distance-Oriented Evaluation with ILS

Require: Initial customer plan $P_0 = [0, c_1, \dots, c_n]$, charging stations S

Ensure: Best executed route under the distance-oriented policy

```
1: Define evaluator  $\mathcal{E}_{\text{dist}}(P)$ :
   follow the first remaining customer in  $P$ 
   if the move is unsafe, go to the nearest reachable
   charging station
   if already at a station and charging is needed,
   charge to full
   update travel, service, waiting, and charging com-
   ponents online
2: Define objective  $f_{\text{dist}}(r) = D(r)$ 
3:  $(P^*, r^*) \leftarrow \text{ILS}(P_0, \mathcal{E}_{\text{dist}}, f_{\text{dist}})$ 
4: return  $(P^*, r^*)$ 
```

cannot continue safely, it moves to the nearest reachable charging station in distance. If the vehicle is already at a charging station and still cannot continue safely, it first charges to full battery. If charging is unnecessary because the battery is already full, the policy moves to the nearest alternative reachable station so that execution does not stall at the current location. In all such cases, station choice is based only on reachability and distance; waiting delay and downstream route timing are not explicitly optimized.

After each executed leg, the simulator updates travel time, battery consumption, service time, station waiting time, and charging duration. If a visited node is an unserved customer, that customer is removed from both the remaining set and the working plan. When all customers have been served, the same distance-oriented feasibility logic is reused for the return to the depot.

Inside the ILS layer, Algorithm 1 uses the full distance-oriented simulator as its evaluator. The objective passed to ILS is traveled distance, not completion time. Thus, ILS improves customer order only insofar as that order yields a shorter executed route under the distance-oriented policy. This distinction matters: Algorithm 1 shares the same search machinery as Algorithm 2, but its search is guided by a distance-oriented evaluator-objective pair.

4.3 Algorithm 2: Delay-Aware Rollout Evaluation with ILS

Algorithm 2 uses the same initial-plan constructor, the same simulator state, and the same ILS machinery, but replaces distance-oriented execution by a rollout-based online evaluator. The central idea is that the next move should be chosen according to predicted remaining completion time rather than immediate geometric proximity.

At each customer decision step, the algorithm looks at a rolling prefix of the current plan. This prefix has length W ,

called the rollout window. From that window, the method forms a candidate subset of size at most K , where $K \leq W$. If $K < W$, candidates are prefiltered by one-step travel time so that only the most promising near-term customers are rollout-evaluated. For each candidate customer c , the algorithm creates a temporary plan with c moved to the front and calls a simulation routine that estimates the remaining completion time under the same dynamic rules as the actual execution model. We denote this estimate by

$$\hat{T}_{\text{rem}}(x; \pi),$$

that is, the simulated remaining completion time from state x under customer plan π . The candidate score is the value of this estimate after moving c to the front of the current plan. The candidate with minimum predicted completion time is selected as the next customer.

Charging-station choice is also delay-aware. Whenever the chosen customer is not safely reachable, the algorithm evaluates every reachable charging station by combining four terms: travel time to the station, waiting time upon arrival, charging duration, and downstream continuation time after leaving that station. In the implementation, the score of a reachable station s is

$$\tau_{is}(t) + w_s(\eta_s) + c(b - e_{is}) + \hat{T}_{\text{rem}}(x_s^+; \pi),$$

where η_s is the arrival time at s and x_s^+ is the post-charge state at s . Thus, “downstream continuation time” means the simulated remaining completion time from the state immediately after arriving, waiting, and fully recharging at the candidate station. The station with the smallest predicted score is selected. Consequently, Algorithm 2 reasons about both immediate charging delay and downstream temporal consequences, whereas Algorithm 1 only uses nearest-station distance.

The return-to-depot phase uses the same rollout-based station-selection logic. Thus, Algorithm 2 is a simulation-driven online routing policy from start to finish, not only during the customer-serving phase.

Inside ILS, Algorithm 2 uses the rollout simulator as its evaluator and total completion time as its objective. A customer-order change is therefore considered beneficial only if it reduces the full executed time after dynamic traffic, queueing, charging, and service effects are accounted for. This is the key methodological difference from Algorithm 1.

Taken together, the unified implementation should be interpreted as a controlled comparison between two evaluator-objective combinations on top of the same customer-plan search mechanism. Algorithm 1 asks ILS to search for a customer order that executes well under a distance-oriented policy. Algorithm 2 asks ILS to search for a customer order that executes well under a delay-aware rollout policy. This separation is what makes the empirical comparison in the experiments meaningful.

Algorithm 2 Delay-Aware Rollout Evaluation with ILS

Require: Initial customer plan $P_0 = [0, c_1, \dots, c_n]$, charging stations S , rollout window W

Ensure: Best executed route under the rollout policy

- 1: Define evaluator $\mathcal{E}_{\text{roll}}(P)$:
 - 2: **while** unserved customers remain **do**
 - 3: build a candidate set from the first W customers of P
 - 4: estimate remaining completion time for each candidate by forward simulation
 - 5: choose the candidate with minimum predicted completion time
 - 6: **if** the chosen customer is unsafe to serve next **then**
 - 7: evaluate all reachable charging stations by travel + wait + charge + downstream score
 - 8: move to the best predicted station and charge
 - 9: **else**
 - 10: move to the chosen customer and serve it
 - 11: **end if**
 - 12: update route state and remove served customers from P
 - 13: **end while**
 - 14: complete the return-to-depot phase with the same delay-aware station logic
 - 15: Define objective $f_{\text{roll}}(r) = T(r)$
 - 16: $(P^*, r^*) \leftarrow \text{ILS}(P_0, \mathcal{E}_{\text{roll}}, f_{\text{roll}})$
 - 17: **return** (P^*, r^*)
-

5 Experimental Results

5.1 Experimental design

All experiments were executed in Python on a laptop with Intel Core i5-13420H and 16GB RAM. We compare two methods under the same EV simulator, congestion model, charging model, and ILS framework. The first method is Alg1, a distance-oriented heuristic with ILS. The second method is Alg2, a delay-aware rollout policy with ILS.

The benchmark contains 8 synthetic datasets with customer counts

$$N \in \{20, 40, 60, 80\},$$

and with two spatial distributions for each size, namely uniform and multicluster. The number of charging stations is fixed at 10% of the customer count, and customer service time is fixed at 5 minutes in all instances. For compact presentation, we denote the uniform instance with N customers by $\text{synth}N_U$ and the multicluster instance with N customers by $\text{synth}N_M$. For example, $\text{synth}20_U$ denotes the uniform dataset with 20 customers and $\text{synth}60_M$ denotes the multicluster dataset with 60 customers.

The exact experimental configuration used in the reported results is as follows. The vehicle starts at 07:00. Battery capacity is 30 kWh, the minimum reserve is 3 kWh, the energy-consumption rate is 0.2 kWh per km,

and charging is performed to full battery at a rate induced by the 50 kW charger model used in the simulator. The congested-edge density is fixed at 0.8. In the multi-seed experiment, the congestion realization is held fixed across seeds, so the observed variation comes from heuristic search rather than from a different underlying traffic graph.

Time-dependent road congestion is modeled by three speed regimes on the sampled congested edges. Peak periods are 07:00–09:00 and 16:30–19:00, with speed fixed at 20 km/h. Normal periods are 05:30–07:00, 09:00–16:30, and 19:00–22:00, with speed fixed at 40 km/h. Off-peak periods are 22:00–05:30, with speed fixed at 60 km/h. Edges outside the congested-edge set always use the off-peak speed of 60 km/h. Hence the network is only partially time-sensitive rather than uniformly congested.

Charging stations are also time-dependent, but under a different rule. Station waiting time is activated only during the interval 15:00–21:30. Outside this interval, waiting time is zero. Inside this interval, each station uses its dataset-specific waiting parameter. The planner is therefore exposed to two distinct temporal effects: travel-time variation on congested edges and waiting-time variation at charging stations.

The stability study uses three search seeds. The initial customer permutation is generated by a depot-fixed OR-Tools construction phase. For Alg2, the multi-seed experiment does not retune the rollout setting. Instead, it imports the best tuned window size W^* from the batch run, while the candidate-set size is fixed deterministically at $K = 0.5W$. In the reported experiments, all selected W^* values are even, so this rule is implemented without rounding ambiguity. Thus, the multi-seed study evaluates stability at a fixed tuned setting rather than mixing stability analysis with parameter tuning.

The ILS and local-search budgets scale adaptively with instance size. In the reported runs, Alg1 uses 10, 14, 21, and 28 outer ILS iterations for $N = 20, 40, 60, 80$, whereas Alg2 uses 10, 18, 27, and 36. The shared local-search layer uses 4, 5, 7, and 10 rounds, with 8, 8, 12, and 16 sampled neighbors per round, respectively. The perturbation strength is set to 0.15 of the customer count rounded to the nearest integer, which gives 3, 6, 9, and 12 customer perturbations for the four benchmark sizes. The initial acceptance temperature is 0.03, and the cooling factor is 0.98.

The evidence in this section is built from two complementary experiment layers. First, a tuned batch run is used to identify the best rollout window W^* for each dataset, with W swept as a percentage of N and K kept as the fixed rule-based fraction of W . Second, a three-seed study is conducted under the fixed configuration above. The first layer answers which rollout window is best on a given instance, while the second layer answers whether the observed behavior is stable with respect to heuristic-search randomness. Because the benchmark consists of syn-

thetic instances and both methods are implemented within the same simulator framework, the experiments should be interpreted as a controlled simulator-based comparison rather than as a comprehensive external benchmark against real-network EVRP methods. Figure 1 shows representative examples of the synthetic spatial layouts used in the experiments.

5.2 Best-case tuned comparison

The primary performance measure is total completion time T in minutes. We also report traveled distance D in kilometers. Method comparison is defined as

$$\Delta T = T_{\text{Alg2}} - T_{\text{Alg1}}, \quad \Delta D = D_{\text{Alg2}} - D_{\text{Alg1}},$$

so that $\Delta T < 0$ means Alg2 is faster than Alg1.

We intentionally separate tuned best-case evidence from fixed-setting evidence. Table 3 reports the best result obtained for Alg2 after sweeping the rollout window on each dataset. These values come from the tuned batch run and therefore represent the strongest single-instance comparison between the two final methods. They are descriptive best-case results rather than the main fixed-setting evidence used later in Table 4.

The tuned comparison yields four descriptive observations. First, in this best-case tuned view, Alg2 yields lower values of the operational objective on all 8 datasets. The mean improvement is -45.20 minutes, the mean relative improvement is -7.76% , and the cumulative time saving over the benchmark is 361.62 minutes. This suggests that, on the tested synthetic benchmark, the delay-aware rollout policy is often more closely aligned with the actual decision objective of the problem.

Second, Alg2 is faster in the reported tuned runs even when it travels farther. In the tuned results, Alg2 increases distance on 5 of the 8 datasets, with mean $\Delta D = +12.81$ km, while still reducing total time on all 8 datasets. This pattern suggests that the delay-aware rollout policy is willing to trade distance for time when completion time depends on traffic regime, station waiting, and downstream battery state rather than on distance alone.

Third, within the tested synthetic benchmark, the size of the time gain appears to depend on instance structure. The mean time improvement is -40.13 minutes for uniform instances and -50.27 minutes for multicluster instances. At the same time, the mean distance penalty is larger in uniform instances ($+18.62$ km) than in multicluster instances ($+7.00$ km). This suggests that clustered layouts may provide more room for time-aware routing decisions, likely because clustered spatial structure creates more pronounced bottlenecks and timing-sensitive decisions.

Fourth, the observed relative gain becomes smaller as N grows under the current computational budget. The largest relative gains appear at $N = 20$, while the gains are smaller for $N = 40, 60, 80$. This trend should not be

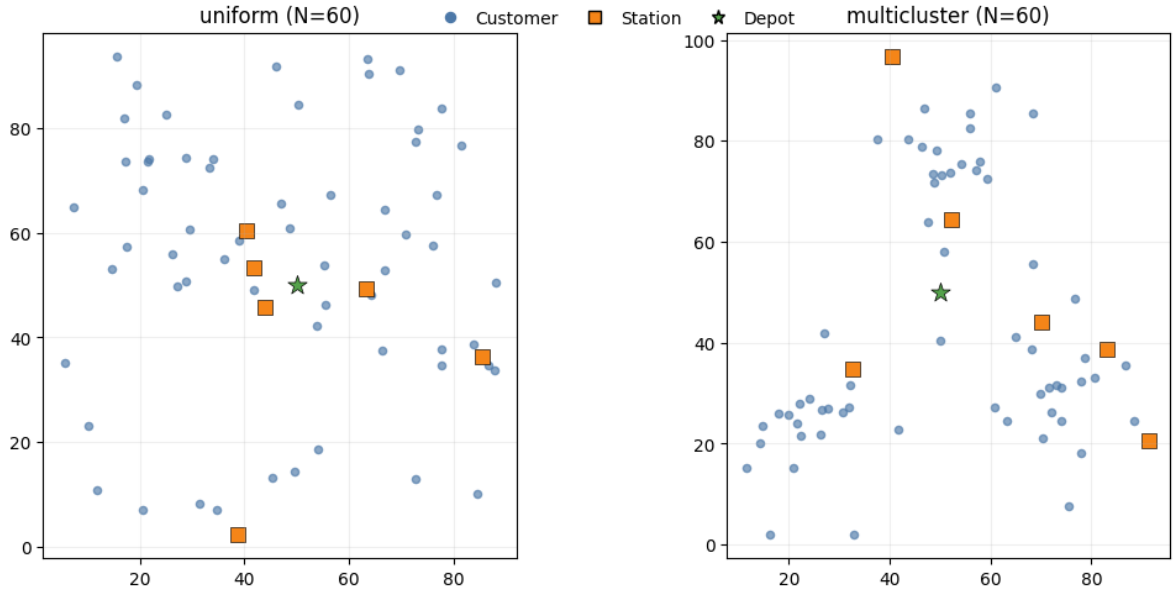


Figure 1: Examples of synthetic spatial distributions used in the experiments.

Table 3: Best-case per-instance results after tuning Alg2 over the tested W values.

Dataset	W^*	T_{Alg1}	T_{Alg2}	ΔT	ΔT (%)	D_{Alg1}	D_{Alg2}
synth20_U	10	473.46	401.27	-72.19	-15.25%	206.0	196.0
synth20_M	6	480.27	373.55	-106.72	-22.22%	207.5	187.5
synth40_U	12	626.48	605.23	-21.25	-3.39%	245.5	271.5
synth40_M	16	545.41	508.24	-37.17	-6.82%	187.5	206.5
synth60_U	24	926.24	870.57	-55.67	-6.01%	320.5	374.0
synth60_M	30	733.85	706.54	-27.31	-3.72%	234.5	232.5
synth80_U	16	1014.74	1003.33	-11.41	-1.12%	340.0	345.0
synth80_M	32	839.64	809.74	-29.90	-3.56%	233.5	264.5

interpreted as evidence that rollout loses conceptual value at larger scale. Rather, it indicates that, under the present sweep and budget, larger instances may require more simulation and more search effort before the same percentage advantage can be realized.

These tuned results motivate the fixed-setting analysis below, but they are not the main basis for the comparative claim because they condition on per-instance selection of W^* . Figure 2 summarizes the cross-dataset relation between time gain and distance change in this tuned view.

5.3 Fixed-setting stability, paired inference, and ablation

The tuned batch run shows the best result on each dataset, but it does not reveal how sensitive the algorithms are to seed-dependent search behavior. To address that issue, we ran a three-seed study using the tuned W^* values from Table 3, while keeping the candidate-set size under the same fixed rule-based relation to W . We treat this fixed-setting three-seed study as the main comparative evidence because it evaluates both methods at fixed tuned

settings rather than re-optimizing the rollout window inside the seed comparison. At the same time, because the congested-edge realization is held fixed, this study should be interpreted as a search-stability analysis under a fixed environment rather than as a robustness test across changing traffic realizations.

Table 4 summarizes the three-seed comparison for the final methods Alg1 and Alg2. The table reports mean and standard deviation over seeds, together with mean distance difference and the coefficient of variation (CV) of Alg2 total completion time across the three seeds.

To complement the dataset-level means in Table 4, Table 5 reports paired inference on the 24 observed ΔT values from the fixed-setting study. The one-sided alternative is that Alg2 is faster than Alg1, that is, $\Delta T < 0$. Because all 24 paired differences are negative, the exact one-sided sign test and the exact Wilcoxon signed-rank test coincide numerically; we report the sign test together with a non-parametric bootstrap confidence interval for the mean.

The fixed-setting study supports four additional observations. First, Alg2 is faster than Alg1 in all 24 reported runs. The mean pairwise difference is -42.46 minutes

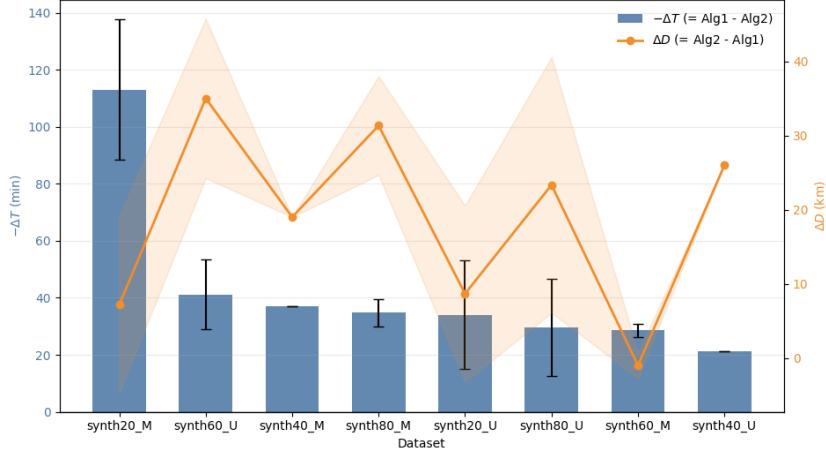


Figure 2: Mean time gain of Alg2 over Alg1 together with the corresponding distance change across datasets.

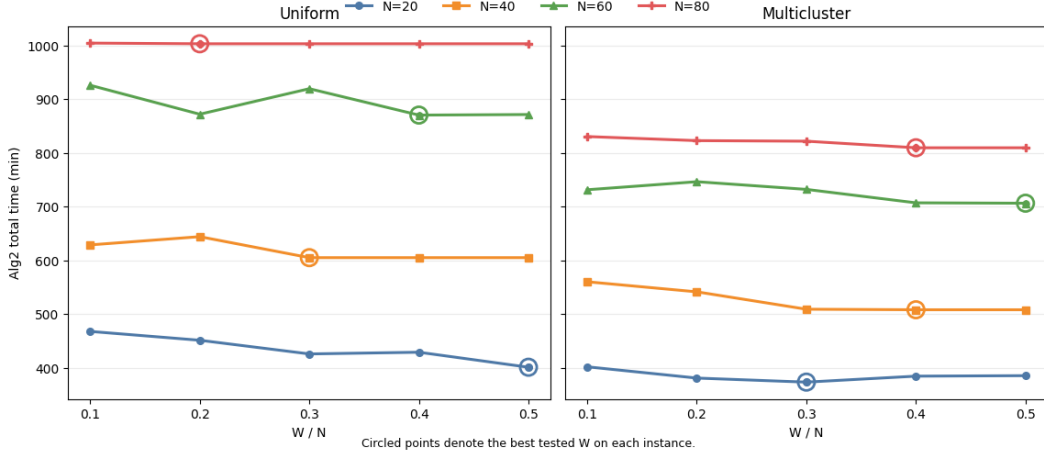


Figure 3: Sensitivity of Alg2 to the rollout window.

with standard deviation 30.00 minutes, and the bootstrap 95% confidence interval for the mean stays strictly below zero. Under the present fixed-instance, fixed-congestion study, this indicates that the observed time gap in favor of Alg2 is not a fragile one-seed artifact.

Second, Alg2 is not only lower on average, but also reasonably stable. The coefficient of variation of Alg2 total time ranges from 0.00% to 4.07%, and in 6 of the 8 datasets it is below 1.5%. This suggests that once W^* and $K = 0.5W^*$ are fixed, Alg2 typically converges to a narrow band of performance under the current experimental setting.

Third, the ablation evidence indicates that the measured ILS gain is larger for Alg2 than for Alg1. Table 6 makes this comparison explicit. Relative to the raw distance-oriented policy, Alg1 reduces total time by only 3.53 minutes on average and improves in only 7 of 24 runs. By contrast, relative to rollout-only, Alg2 reduces time by 11.77 minutes on average and improves in 15 of 24 runs. This means the final performance gap cannot be attributed mainly to the outer metaheuristic. Rather, un-

der the present benchmark, the larger gains appear when the search layer is coupled with a time-aware evaluator. Figure 4 visualizes both the ablation comparison and the associated runtime contrast.

Fourth, Alg2 achieves lower completion times at a substantial computational cost. Across the 24 runs, the mean runtime of Alg1 is about 0.21 seconds, whereas the mean runtime of Alg2 is about 105.58 seconds. Thus, the empirical time savings of Alg2 should be interpreted as a quality–cost trade-off rather than unconditional dominance. Figure 5 shows how runtime grows with instance size for the tested methods.

For the representative instance synth60_U, the tuned single-run result shows that Alg2 reduces total time from 926.24 to 870.57 minutes while increasing distance from 320.5 to 374.0 km. Figure 6 helps interpret this result at the component level. The breakdown shows that the gain comes from lower drive-time exposure and slightly lower charging-related overhead, not from reducing service or eliminating charging. This is consistent with the interpretation that the main mechanism of Alg2 is temporal syn-

Table 4: Fixed-setting three-seed comparison at the tuned W^* values.

Dataset	W^*	T_{Alg1}	T_{Alg2}	Mean ΔT	Mean ΔD	CV(T_{Alg2}) (%)
synth20_U	10	461.33 $\pm$ 21.02	427.26 $\pm$ 3.49	-34.07	8.67	0.82
synth20_M	6	494.99 $\pm$ 13.49	381.94 $\pm$ 15.54	-113.05	7.17	4.07
synth40_U	12	626.48 $\pm$ 0.00	605.23 $\pm$ 0.00	-21.25	26.00	0.00
synth40_M	16	545.41 $\pm$ 0.00	508.24 $\pm$ 0.00	-37.17	19.00	0.00
synth60_U	24	907.06 $\pm$ 5.49	865.85 $\pm$ 12.79	-41.21	35.00	1.48
synth60_M	30	733.85 $\pm$ 0.00	705.20 $\pm$ 2.33	-28.65	-1.00	0.33
synth80_U	16	1058.27 $\pm$ 11.94	1028.71 $\pm$ 6.19	-29.56	23.33	0.60
synth80_M	32	850.49 $\pm$ 5.64	815.77 $\pm$ 0.99	-34.72	31.33	0.12

Table 5: Paired inference for ΔT in the fixed-setting three-seed study.

Statistic	Value
Number of paired runs	24
Runs with $\Delta T < 0$	24 / 24
Mean ΔT (min)	-42.46
Median ΔT (min)	-37.17
Bootstrap 95% CI for mean ΔT	$[-55.23, -31.97]$
Exact one-sided sign test p	5.96×10^{-8}

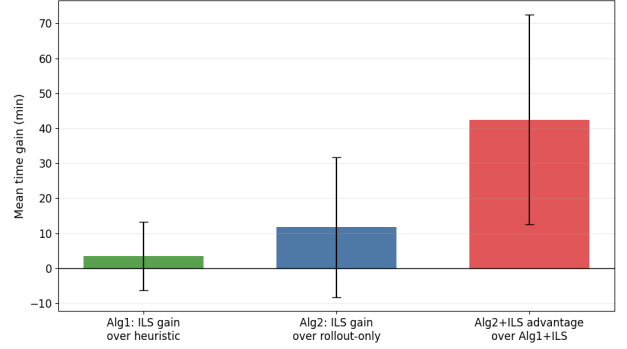
Table 6: Ablation summary from the fixed-setting three-seed study. Negative ΔT favors the first-named method.

Comparison	Mean ΔT (min)	Std. dev.	Improve / Tie / Worse
Alg1 + ILS minus raw distance policy	-3.53	9.74	7 / 15 / 2
Alg2 + ILS minus rollout-only	-11.77	19.99	15 / 9 / 0
Alg2 + ILS minus Alg1 + ILS	-42.46	30.00	24 / 0 / 0

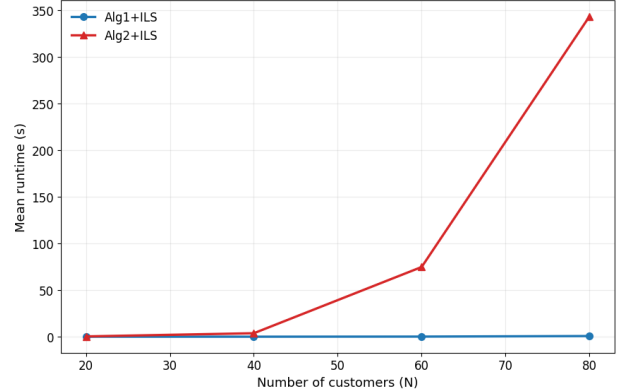
chronization rather than workload reduction.

The sensitivity analysis leads to one further conclusion. Figure 3 shows the corresponding response of Alg2 to the rollout window. The selected best ratios W^*/N range from 0.2 to 0.5 across the benchmark. This indicates that there is no single globally optimal rollout window for all tested synthetic instances. Small windows are often too myopic to capture downstream charging and congestion effects, whereas very large windows increase simulation burden and may introduce comparisons whose marginal value is limited.

Taken together, the results support three bounded conclusions. First, on the tested synthetic benchmark and under the present tuning and budget settings, Alg2 tends to yield lower completion time than Alg1, with the main fixed-setting evidence coming from Table 4 and Table 5. Second, this time difference is stable with respect to search seed under the current fixed congestion realization. Third, the ablation evidence suggests that the observed gain is associated more with the interaction between roll-



(a) Mean time gain from ILS for Alg1 and Alg2, together with the overall time difference between the two final methods.



(b) Mean runtime comparison between Alg1 and Alg2 in the three-seed study.

Figure 4: Ablation and runtime evidence from the fixed-setting three-seed study.

out and ILS than with ILS alone.

At the same time, the scope of the evidence should be stated clearly. The datasets are synthetic instances rather than real road networks. The multi-seed study keeps the congested-edge realization fixed, so it measures algorithmic stability under a fixed environment rather than robustness to changing environments. The runtime and sensitivity conclusions are also conditional on the present ILS budget and rollout sweep. These limitations do not weaken the internal logic of the findings, but they do define the boundary of the claims supported by the current experiments.

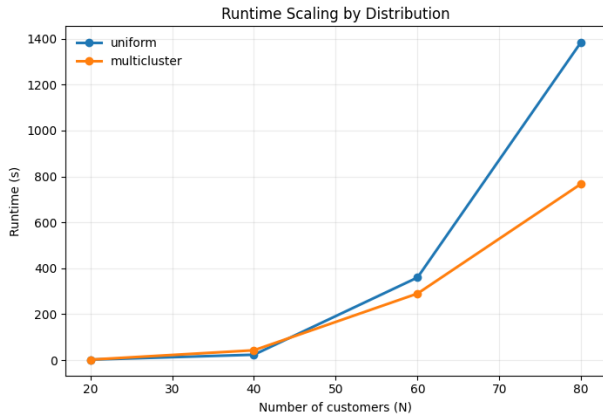


Figure 5: Runtime scaling with respect to instance size.

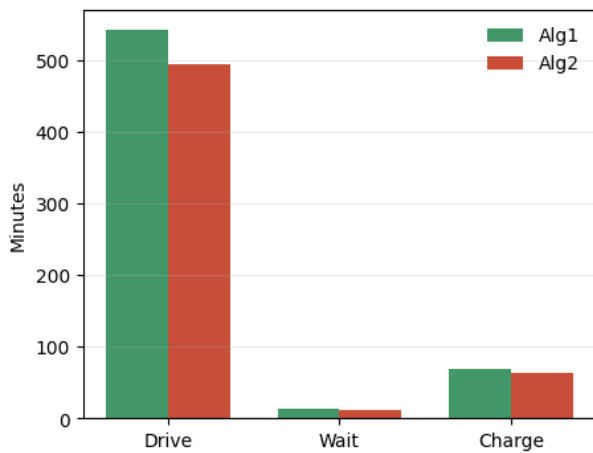


Figure 6: Component-wise time breakdown on synth60_U.

6 Conclusion and Future Work

This paper studied a single-vehicle electric vehicle routing problem with time-dependent travel time and time-dependent charging-station delay. Two online policies were compared under the same simulator and ILS framework: a distance-oriented baseline and a delay-aware rollout policy.

The results suggest that the delay-aware rollout policy is more closely aligned with the operational objective of minimizing realized completion time under the present experimental setting. The fixed-setting three-seed study, which is the main comparative evidence in the paper, shows that Alg2 is faster in all 24 observed paired runs, with mean $\Delta T = -42.46$ minutes and a bootstrap 95% confidence interval of $[-55.23, -31.97]$ minutes. The best-case tuned sweep in Table 3 points in the same direction, but it should be interpreted as descriptive tuned evidence rather than as the main comparative claim. On the tested synthetic benchmark, these results indicate that shorter distance does not necessarily imply shorter operational time under time-varying traffic and charging de-

lay. The ablation results further suggest that the main performance gain is associated with the interaction between rollout-based temporal evaluation and the outer search layer. These findings should be interpreted as benchmark evidence from a controlled simulator-based study on synthetic instances, not as a general claim about EV routing performance in real transportation systems.

The current study is limited to synthetic instances, a controlled comparison within a shared framework rather than a broad benchmark against external baselines, a fixed congestion realization in the multi-seed analysis, and a single-vehicle setting with deterministic charging delay profiles. Future work will extend the framework to GIS-based or real road-network data, broader benchmark sets and external comparator methods, stochastic traffic and charging environments, and multi-vehicle EV routing. Another important direction is to design faster approximation mechanisms for rollout so that similar quality gains can be pursued at lower computational cost.

Statements and Declarations

Competing Interests The authors declare that there are no competing financial or non-financial interests related to this work.

References

1. Adamo, T., Gendreau, M., Ghiani, G., Guerriero, E.: A review of recent advances in time-dependent vehicle routing. *European Journal of Operational Research* **319**(1), 1–15 (2024). DOI 10.1016/j.ejor.2024.06.016
2. Bertazzi, L., Secomandi, N.: Faster rollout search for the vehicle routing problem with stochastic demands and restocking. *European Journal of Operational Research* **270**(2), 487–497 (2018). DOI 10.1016/j.ejor.2018.03.034
3. Bertsekas, D.P.: *Dynamic Programming and Optimal Control*, vol. 2, 4th edn. Athena Scientific, Belmont, MA (2010)
4. Clarke, G., Wright, J.W.: Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research* **12**(4), 568–581 (1964). DOI 10.1287/opre.12.4.568
5. Conrad, R.G., Figliozzi, M.A.: The recharging vehicle routing problem. In: T. Doolen, E.V. Aken (eds.) *Proceedings of the 2011 Industrial Engineering Research Conference*, pp. 1–8 (2011)
6. Dabia, S., Röpke, S., Woensel, T.V., de Kok, T.: Branch and price for the time-dependent vehicle rout-

- ing problem with time windows. *Transportation Science* **47**(3), 380–396 (2013). DOI 10.1287/trsc.1120.0443
7. Dantzig, G.B., Ramser, J.H.: The truck dispatching problem. *Management Science* **6**(1), 80–91 (1959). DOI 10.1287/mnsc.6.1.80
8. Erdelić, T., Đurasević, M., Erdelić, M., Carić, T.: Time-dependent electric vehicle routing problem with time windows and charging time dependent on the state of charge. *Computers & Industrial Engineering* **213**, 111758 (2025). DOI 10.1016/j.cie.2025.111758
9. Figliozzi, M.A.: The time dependent vehicle routing problem with time windows: Benchmark problems, an efficient solution algorithm, and solution characteristics. *Transportation Research Part E: Logistics and Transportation Review* **48**(3), 616–636 (2012). DOI 10.1016/j.tre.2011.11.006
10. Froger, A., Mendoza, J.E., Jabali, O., Laporte, G.: Improved formulations and algorithmic components for the electric vehicle routing problem with nonlinear charging functions. *Computers & Operations Research* **104**, 256–294 (2019). DOI 10.1016/j.cor.2018.12.013
11. Ichoua, S., Gendreau, M., Potvin, J.Y.: Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research* **144**(2), 379–396 (2003). DOI 10.1016/S0377-2217(02)00147-9
12. Keskin, M., Akhavan-Tabatabaei, R., Çatay, B.: Electric vehicle routing problem with time windows and stochastic waiting times at recharging stations. In: *Proceedings of the 2019 Winter Simulation Conference (WSC)*, pp. 1649–1659 (2019). DOI 10.1109/WSC40007.2019.9004766
13. Keskin, M., Çatay, B.: Partial recharge strategies for the electric vehicle routing problem with time windows. *Transportation Research Part C: Emerging Technologies* **65**, 111–127 (2016). DOI 10.1016/j.trc.2016.01.013
14. Keskin, M., Çatay, B., Laporte, G.: A simulation-based heuristic for the electric vehicle routing problem with time windows and stochastic waiting times at recharging stations. *Computers & Operations Research* **125**, 105060 (2021). DOI 10.1016/j.cor.2020.105060
15. Keskin, M., Laporte, G., Çatay, B.: Electric vehicle routing problem with time-dependent waiting times at recharging stations. *Computers & Operations Research* **107**, 77–94 (2019). DOI 10.1016/j.cor.2019.02.014
16. Kool, W., van Hoof, H., Welling, M.: Attention, learn to solve routing problems! In: *International Conference on Learning Representations (ICLR)* (2019). URL <https://openreview.net/forum?id=ByxBFsRqYm>
17. Laporte, G.: The vehicle routing problem: An overview of exact and approximate algorithms. *European Journal of Operational Research* **59**(3), 345–358 (1992). DOI 10.1016/0377-2217(92)90192-C
18. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search: Framework and applications. In: M. Gendreau, J.Y. Potvin (eds.) *Handbook of Metaheuristics*, pp. 129–168. Springer, Cham (2019). DOI 10.1007/978-3-319-91086-4_5
19. Malandraki, C., Daskin, M.S.: Time dependent vehicle routing problems: Formulations, properties and heuristic algorithms. *Transportation Science* **26**(3), 185–200 (1992). DOI 10.1287/trsc.26.3.185
20. Mladenović, N., Hansen, P.: Variable neighborhood search. *Computers & Operations Research* **24**(11), 1097–1100 (1997). DOI 10.1016/S0305-0548(97)00031-2
21. Montoya, A., Guéret, C., Mendoza, J.E., Villegas, J.G.: The electric vehicle routing problem with nonlinear charging function. *Transportation Research Part B: Methodological* **103**, 87–110 (2017). DOI 10.1016/j.trb.2017.02.004
22. Mor, A., Speranza, M.G.: Vehicle routing problems over time: A survey. *Annals of Operations Research* **314**(1), 255–275 (2022). DOI 10.1007/s10479-021-04488-0
23. Nazari, M., Oroojlooy, A., Snyder, L.V., Takáč, M.: Reinforcement learning for solving the vehicle routing problem. In: *Advances in Neural Information Processing Systems* 31, pp. 9861–9871 (2018)
24. Novoa, C., Storer, R.: An approximate dynamic programming approach for the vehicle routing problem with stochastic demands. *European Journal of Operational Research* **196**(2), 509–515 (2009). DOI 10.1016/j.ejor.2008.03.023
25. Pillac, V., Gendreau, M., Guéret, C., Medaglia, A.L.: A review of dynamic vehicle routing problems. *European Journal of Operational Research* **225**(1), 1–11 (2013). DOI 10.1016/j.ejor.2012.08.015
26. Psaraftis, H.N., Wen, M., Kontovas, C.A.: Dynamic vehicle routing problems: Three decades and counting. *Networks* **67**(1), 3–31 (2016). DOI 10.1002/net.21628

27. Rios, B.H.O., Xavier, E.C., Miyazawa, F.K., Amorim, P., Curcio, E., Santos, M.J.: Recent dynamic vehicle routing problems: A survey. *Computers & Industrial Engineering* **160**, 107604 (2021). DOI 10.1016/j.cie.2021.107604
28. Ropke, S., Pisinger, D.: An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science* **40**(4), 455–472 (2006). DOI 10.1287/trsc.1050.0135
29. Schneider, M., Stenger, A., Goeke, D.: The electric vehicle-routing problem with time windows and recharging stations. *Transportation Science* **48**(4), 500–520 (2014). DOI 10.1287/trsc.2013.0490
30. Secomandi, N.: A rollout policy for the vehicle routing problem with stochastic demands. *Operations Research* **49**(5), 796–802 (2001). DOI 10.1287/opre.49.5.796.10608
31. Solomon, M.M.: Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research* **35**(2), 254–265 (1987). DOI 10.1287/opre.35.2.254
32. Verma, A.: Electric vehicle routing problem with time windows, recharging stations and battery swapping stations. *EURO Journal on Transportation and Logistics* **7**(4), 415–451 (2018). DOI 10.1007/s13676-018-0136-9
33. Vidal, T.: Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research* **140**, 105643 (2022). DOI 10.1016/j.cor.2021.105643
34. Zhang, J., Woensel, T.V.: Dynamic vehicle routing with random requests: A literature review. *International Journal of Production Economics* **256**, 108751 (2023). DOI 10.1016/j.ijpe.2022.108751
35. Zhou, S., Zhang, D., Ji, B., Zhou, S., Li, S., Zhou, L.: A milp model and heuristic method for the time-dependent electric vehicle routing and scheduling problem with time windows. *Journal of Cleaner Production* **434**, 140188 (2024). DOI 10.1016/j.jclepro.2023.140188